EE-170L Computer Programming Lab

Laboratory Report — No. 14

Relative Grading System in C++

Student Name	Muhammad Adil
Registration No.	BF25NWELE0669
Course	EE-170L – Computer Programming Lab
Lab No.	14
Topic	Relative Grading: Normalization, Grade Assignment, GPA
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand the UET Peshawar relative grading mechanism and why it is used instead of absolute marking.
- Implement normalization of raw marks using the formula: Normalized = (Actual / Highest) × 100.
- Map normalized marks to letter grades across ten equal intervals from 50 to 100.
- Convert letter grades to GPA points on the standard 4.0 scale.
- Use arrays to store and process data for an entire class of students.
- Display a formatted result table with class statistics and grade distribution.

2. Theoretical Background

2.1 What is Relative Grading?

Relative grading (also called grading on a curve) assigns letter grades based on a student's performance relative to the rest of the class rather than against a fixed percentage scale. There is no fixed score that earns an "A" or a "B" — the grade is an expression of how well one student performed compared to their peers.

UET Peshawar uses relative grading for Type-A (theory) courses and absolute grading for Type-B (lab/practical) courses. Under the relative system, the highest scorer in the class is always normalized to 100, and all other marks are scaled accordingly.

2.2 Normalization Formula

The key formula used by UET Peshawar is:

```
Normalized Marks = ( Actual Marks / Highest Marks in Class ) x 100
```

This guarantees that the top scorer always receives a normalized mark of 100, regardless of how difficult the exam was. All other students are scaled proportionally.

Example: If the highest mark in a class of 30 students is 78 and a student scored 60, their normalized mark = (60 / 78) x 100 = 76.92. That maps to grade B-.

2.3 Grade Thresholds (Normalized Marks)

The range 50–100 is divided into ten equal intervals of 5 marks each. Students with normalized marks below 50 receive an F. The student who achieves the maximum threshold (100) receives an A+.

Normalized Marks	Letter Grade	Grade Points	Normalized Marks	Letter Grade
= 100	A+	4.00	70 – 74.99	C+
95 – 99.99	A	4.00	65 – 69.99	C
90 – 94.99	A-	3.67	60 – 64.99	C-
85 – 89.99	B+	3.33	55 – 59.99	D+
80 – 84.99	B	3.00	50 – 54.99	D
75 – 79.99	B-	2.67	< 50	F

2.4 Grade Point Average (GPA)

Each letter grade corresponds to a fixed grade point on the 4.0 scale. A and A+ both carry 4.00 points; A+ is an honorary distinction for the class topper but does not raise the GPA above 4.00. The minimum passing grade is D (1.00). A student who fails (F, 0.00) must re-register in the course to earn credit.

3. Example Programs

3.1 Single-Student Grade Calculator

This minimal example demonstrates normalization and grade assignment for one student, illustrating the core mathematical steps before scaling to a full class.

```
#include <iostream>
#include <string>
using namespace std;

string assignGrade(float nm) {
    if (nm >= 100) return "A+";
    if (nm >= 95)  return "A";
    if (nm >= 90)  return "A-";
    if (nm >= 85)  return "B+";
    if (nm >= 80)  return "B";
    if (nm >= 75)  return "B-";
    if (nm >= 70)  return "C+";
    if (nm >= 65)  return "C";
    if (nm >= 60)  return "C-";
    if (nm >= 55)  return "D+";
    if (nm >= 50)  return "D";
    return "F";
}

int main() {
    float actual, highest;
    cout << "Enter student marks : "; cin >> actual;
    cout << "Enter highest marks  : "; cin >> highest;
    float nm = (actual / highest) * 100;
    cout << "Normalized : " << nm << endl;
    cout << "Grade      : " << assignGrade(nm) << endl;
    return 0;
}
```

Output:

```
Enter student marks : 74
Enter highest marks  : 92
Normalized : 80.43
Grade      : B
```

3.2 Class Grading with Arrays

This example extends the single-student approach to a full class using arrays and a loop to find the highest mark automatically before normalizing everyone.

```
#include <iostream>
#include <string>
using namespace std;

string assignGrade(float nm) { /* same as above */ }

int main() {
    int n; float marks[50];
    cout << "Enter number of students: "; cin >> n;
    float highest = 0;
    for (int i = 0; i < n; i++) {
        cout << "Marks[" << i+1 << "]: "; cin >> marks[i];
        if (marks[i] > highest) highest = marks[i];
    }
    cout << "\nResult Table:\n";
    for (int i = 0; i < n; i++) {
        float nm = (marks[i] / highest) * 100;
        cout << "Student " << i+1
             << "  Norm=" << nm
             << "  Grade=" << assignGrade(nm) << endl;
    }
    return 0;
}
```

Output:

```
Enter number of students: 3
Marks[1]: 90   Marks[2]: 75   Marks[3]: 45
Result Table:
Student 1  Norm=100.00  Grade=A+
Student 2  Norm=83.33   Grade=B
Student 3  Norm=50.00   Grade=D
```

4. Lab Task & Solution

Task — Full Relative Grading System

Implement a complete relative grading system in C++ for a class of up to 100 students. The program must: accept student names and marks, find the highest mark automatically, normalize all marks, assign letter grades according to the UET Peshawar scheme, convert grades to GPA points, display a formatted result table, and show class statistics including pass/fail count, class average, and a grade distribution chart.

```
// ============================================================
// Lab 14 - Relative Grading System (UET Peshawar)
// Course : EE-170L Computer Programming Lab
// Name   : Muhammad Adil  |  Reg: BF25NWELE0669
// Date   : 19-May-2026
// ============================================================

#include <iostream>   // for cin, cout, endl
#include <iomanip>    // for setw(), setprecision(), fixed
#include <string>     // for string type and comparisons
using namespace std;
```

```
// Maximum number of students the program can handle
const int MAX_STUDENTS = 100;

// ── Function Prototypes ──────────────────────────────
float  normalizeMarks(float actual, float highest);
string assignGrade(float normalized);
float  gradeToGPA(string grade);
void   printLine(char ch, int n);
void   printTable(string names[], float actual[],
                  float normalized[], string grades[],
                  float gpa[], int n);
void   printStats(float actual[], float normalized[],
                  string grades[], float gpa[], int n);

// ── Main Function ────────────────────────────────────
int main() {
    int    n;                       // number of students
    string names[MAX_STUDENTS];     // student names
    float  actual[MAX_STUDENTS];    // raw marks entered by user
    float  normalized[MAX_STUDENTS];// marks scaled to 0-100
    string grades[MAX_STUDENTS];    // letter grade for each student
    float  gpa[MAX_STUDENTS];       // GPA points for each student
    float  highest = 0;             // tracks the highest mark in class

    // Display program header
    printLine('=', 60);
    cout << "   UET PESHAWAR - RELATIVE GRADING SYSTEM\n";
    printLine('=', 60);

    // Read total number of students
    cout << "Enter total number of students: ";
    cin  >> n;

    // Input loop: collect name and marks for each student
    for (int i = 0; i < n; i++) {
        cout << "Student " << i+1 << " Name  : ";
        cin.ignore();           // flush newline left by previous cin >>
        getline(cin, names[i]); // getline() captures names with spaces
        cout << "Student " << i+1 << " Marks : ";
        cin >> actual[i];

        // Validate: marks must be between 0 and 100
        while (actual[i] < 0 || actual[i] > 100) {
            cout << "  Invalid. Enter 0-100: ";
            cin >> actual[i];
        }

        // Update highest mark if this student scored higher
        if (actual[i] > highest) highest = actual[i];
    }

    // Processing loop: normalize, assign grade, convert to GPA
    for (int i = 0; i < n; i++) {
        normalized[i] = normalizeMarks(actual[i], highest); // scale marks
        grades[i]     = assignGrade(normalized[i]);          // map to letter
        gpa[i]        = gradeToGPA(grades[i]);               // map to points
    }

    // Display formatted results table and class statistics
    printTable(names, actual, normalized, grades, gpa, n);
    printStats(actual, normalized, grades, gpa, n);
    return 0;
}

// ── normalizeMarks ───────────────────────────────────
// Formula: Normalized = (Actual / Highest) * 100
// Scales all marks so the top scorer always gets 100
float normalizeMarks(float actual, float highest) {
    if (highest == 0) return 0;   // guard against division by zero
```

```
    return (actual / highest) * 100.0f;
}

// ── assignGrade ──────────────────────────────────────────
// Maps normalized marks to a letter grade.
// Range 50-100 split into 10 equal intervals of 5 marks.
// Below 50 = F (fail).  Exactly 100 = A+ (top scorer).
string assignGrade(float nm) {
    if (nm >= 100) return "A+";  // maximum threshold: class topper
    if (nm >= 95)  return "A";   // 95 - 99.99
    if (nm >= 90)  return "A-";  // 90 - 94.99
    if (nm >= 85)  return "B+";  // 85 - 89.99
    if (nm >= 80)  return "B";   // 80 - 84.99
    if (nm >= 75)  return "B-";  // 75 - 79.99
    if (nm >= 70)  return "C+";  // 70 - 74.99
    if (nm >= 65)  return "C";   // 65 - 69.99
    if (nm >= 60)  return "C-";  // 60 - 64.99
    if (nm >= 55)  return "D+";  // 55 - 59.99
    if (nm >= 50)  return "D";   // 50 - 54.99: minimum passing grade
    return "F";                   // below 50: failed, must repeat course
}

// ── gradeToGPA ───────────────────────────────────────────
// Converts a letter grade string to a GPA point (4.0 scale).
// A and A+ both return 4.00; A+ is an honorary distinction only.
float gradeToGPA(string g) {
    if (g=="A+" || g=="A") return 4.00f;  // excellent
    if (g=="A-") return 3.67f;              // very good
    if (g=="B+") return 3.33f;              // above average (high)
    if (g=="B")  return 3.00f;              // above average
    if (g=="B-") return 2.67f;              // above average (low)
    if (g=="C+") return 2.33f;              // average (high)
    if (g=="C")  return 2.00f;              // average
    if (g=="C-") return 1.67f;              // average (low)
    if (g=="D+") return 1.33f;              // minimum acceptable (high)
    if (g=="D")  return 1.00f;              // minimum acceptable
    return 0.00f;                           // F: failed
}

// ── printLine ────────────────────────────────────────────
// Prints n copies of character ch followed by a newline.
// Used to draw separator lines in the output table.
void printLine(char ch, int n) {
    for (int i = 0; i < n; i++) cout << ch;
    cout << "\n";
}

// ── printTable ───────────────────────────────────────────
// Displays a formatted result table with all student grades.
// setw() pads each column to a fixed width for alignment.
void printTable(string names[], float actual[],
                float normalized[], string grades[],
                float gpa[], int n) {
    // Print column headers
    cout << left
         << setw(4)  << "#"
         << setw(20) << "Name"
         << setw(10) << "Marks"
         << setw(14) << "Normalized"
         << setw(8)  << "Grade"
         << setw(6)  << "GPA" << "\n";
    printLine('-', 60);
    // Print one row per student
    for (int i = 0; i < n; i++) {
        cout << left
             << setw(4)  << i+1
             << setw(20) << names[i]
             << setw(10) << fixed << setprecision(2) << actual[i]
             << setw(14) << normalized[i]
             << setw(8)  << grades[i]
             << setw(6)  << gpa[i] << "\n";
```

```
    }
    printLine('-', 60);
}

// ── printStats ──────────────────────────────────────────
// Calculates and displays class-wide statistics:
// total passed/failed, average raw marks, and average GPA.
void printStats(float actual[], float normalized[],
                string grades[], float gpa[], int n) {
    float sumM = 0, sumN = 0, sumG = 0; // running totals
    int   pass = 0, fail = 0;           // pass/fail counters

    // Single pass: accumulate sums and count pass/fail
    for (int i = 0; i < n; i++) {
        sumM += actual[i];       // total raw marks
        sumN += normalized[i];   // total normalized marks
        sumG += gpa[i];          // total GPA points
        if (grades[i] == "F") fail++;  // F = failed
        else                  pass++;  // any other grade = passed
    }
    // Display computed statistics
    cout << "Passed: " << pass << "  Failed: " << fail << "\n";
    cout << "Class Avg: " << sumM/n << " marks\n"; // mean raw marks
    cout << "Avg GPA  : " << sumG/n << "\n";       // mean GPA
}
```

Sample Output:

```
============================================================
   UET PESHAWAR - RELATIVE GRADING SYSTEM
============================================================
Enter total number of students: 5

#   Name                Marks     Normalized    Grade   GPA
------------------------------------------------------------
1   Ali Khan            78.00     84.78         B       3.00
2   Ahmed Raza          92.00     100.00        A+      4.00
3   Fatima Noor         55.00     59.78         D+      1.33
4   Bilal Mehmood       45.00     48.91         F       0.00
5   Sara Gul            92.00     100.00        A+      4.00
------------------------------------------------------------
Passed: 4  Failed: 1
Class Avg: 72.40 marks
Avg GPA  : 2.47
```

How it works:

- The program first collects all marks and tracks the highest score in one pass through the loop.
- Normalization is applied to every student using the ratio (actual / highest) × 100, so the top scorer always receives 100.
- assignGrade() maps the normalized mark to a letter grade using cascading if-else comparisons in descending order.
- gradeToGPA() uses string comparison to return the corresponding 4.0-scale point value.
- Input validation inside the loop rejects marks outside the 0–100 range before they affect the highest tracker.

5. Relative vs Absolute Grading — Key Differences

Feature	Absolute Grading	Relative Grading
Grade boundary	Fixed (e.g. 85% = A)	Varies with class performance

Top scorer	May or may not get A	Always normalized to 100 (A+)
Exam difficulty	Directly impacts grades	Cancelled out by normalization
Used for	Type-B (Lab) courses	Type-A (Theory) courses
F grade trigger	Below fixed threshold	Normalized marks < 50
GPA scale	4.0 (same)	4.0 (same)

6. Conclusion

This lab implemented the complete UET Peshawar relative grading system in C++. Key takeaways:

- Normalization Formula: Dividing each student's marks by the highest mark and multiplying by 100 ensures the top scorer always maps to 100, regardless of exam difficulty.
- Grade Assignment: The 50–100 normalized range is split into ten 5-mark intervals, each corresponding to one of ten letter grades (D through A). Below 50 is F; at 100 is A+.
- GPA Mapping: Each letter grade carries a fixed point value on the 4.0 scale. A and A+ both yield 4.00 — A+ is an honorary distinction only.
- Arrays and Loops: Storing student data in parallel arrays and processing them in loops is the standard C++ technique for batch data operations of this kind.
- Input Validation: Rejecting out-of-range marks before they enter the calculation prevents corruption of the highest-mark tracker and ensures correct normalization for all students.

Relative grading is fundamentally a statistical tool: it rewards performance above the class average and penalizes performance below it. Mastering its implementation in C++ reinforces array manipulation, function design, and formatted output — skills that underpin every larger software system.